

When Can You Poison Rewards?

A Tight Characterization of Reward Poisoning in Linear MDPs

Jose E. Aguilar Escamilla, Haoyang Hong, Jiawei Li,
Haoyu Zhao, Xuezhou Zhang, Sanghyun Hong, Huazheng Wang

ICML 2026



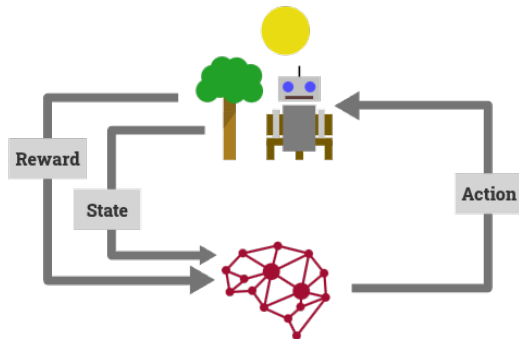
Reinforcement learning, safety, and reward poisoning

Safety-critical decision-making problems like robotics, healthcare, and LLM alignment are increasingly solved by reinforcement learning. Unfortunately, RL can be sensitive to noise and adversarial threats, posing a challenge for real-world trustworthy RL systems.

We study **targeted adversarial reward poisoning**, where an adversary perturbs the feedback/reward signal. Precisely:

Theory when is such an attack even *possible*?

Practice which deployments are inherently safer?



Reinforcement learning interaction loop.

Targeted Reward Poisoning Attacks in RL

Adversary's objective: Make $\pi^\dagger \neq \pi^*$ optimal over **ALL** policies under the relevant state-action distribution.

Reward Poisoning: Find poisoning strategy $\delta_h(s, a)$ to perturb rewards $r_t^\dagger = r_t(s, a) + \delta_t(s, a)$.

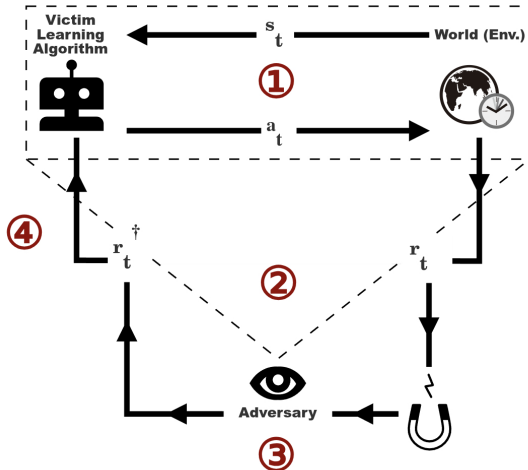
Targeted Reward Poisoning Attacks in RL

Adversary's objective: Make $\pi^\dagger \neq \pi^*$ optimal over **ALL** policies under the relevant state-action distribution.

Reward Poisoning: Find poisoning strategy $\delta_h(s, a)$ to perturb rewards $r_t^\dagger = r_t(s, a) + \delta_t(s, a)$.

- (1) Learner gets s_t , selects a_t .
- (2) Adversary observes: (s_t, a_t, r_t) .
- (3) Reward poisoning: $r_t \rightarrow r_t^\dagger$.
- (4) Learner adjusts policy with $(s_t, a_t, r_t^\dagger)$

Repeat.



An additional paradigm for safe decision-making

Literature on robustness *classically* focuses on developing defenses for *algorithm robustness* and their shortcomings amid stronger attacks. We pursue a complementary lens called **Intrinsic Robustness**.

Algorithmic robustness

Design *learners* that tolerate corruption (robust estimators, detection, defenses).

Safety is a property of the **algorithm**.

An additional paradigm for safe decision-making

Literature on robustness *classically* focuses on developing defenses for *algorithm robustness* and their shortcomings amid stronger attacks. We pursue a complementary lens called **Intrinsic Robustness**.

Algorithmic robustness

Design *learners* that tolerate corruption (robust estimators, detection, defenses).

Safety is a property of the **algorithm**.

Intrinsic robustness

Some *attack objectives* are *impossible* to achieve, even against non-robust learners.

Safety is a property of the **environment**.

An additional paradigm for safe decision-making

Literature on robustness *classically* focuses on developing defenses for *algorithm robustness* and their shortcomings amid stronger attacks. We pursue a complementary lens called **Intrinsic Robustness**.

Algorithmic robustness

Design *learners* that tolerate corruption (robust estimators, detection, defenses).

Safety is a property of the **algorithm**.

Intrinsic robustness

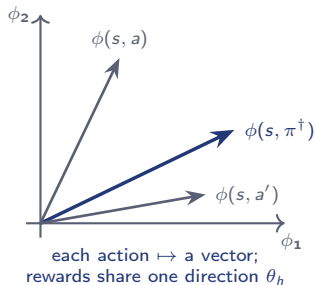
Some *attack objectives* are *impossible* to achieve, even against non-robust learners.

Safety is a property of the **environment**.

Why it happens. **Structural assumptions** about the world correlate rewards together. Making $\pi^\dagger$ optimal requires adjustment of all rewards at once, but “unintended” reward perturbations due to correlation can make the objective **impossible** regardless of budget.

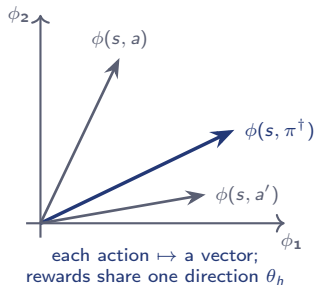
Theoretical Setting: linear MDPs

Each “situation” (state and action) is mapped to a **feature vector** $\phi(s, a) \in \mathbb{R}^d$ ($\|\phi\|_2 \leq 1$).



Theoretical Setting: linear MDPs

Each “situation” (state and action) is mapped to a **feature vector** $\phi(s, a) \in \mathbb{R}^d$ ($\|\phi\|_2 \leq 1$).

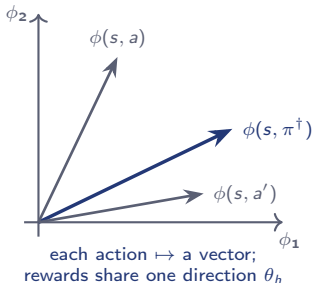


Linear reward and dynamics:

$$\underbrace{r_h(s, a) = \phi(s, a)^\top \theta_h}_{\text{reward}}, \quad \underbrace{P_h(\cdot \mid s, a) = \sum_i \phi_i(s, a) P_h^{(i)}(\cdot)}_{\text{transitions}}.$$

Theoretical Setting: linear MDPs

Each “situation” (state and action) is mapped to a **feature vector** $\phi(s, a) \in \mathbb{R}^d$ ($\|\phi\|_2 \leq 1$).



Intuition:

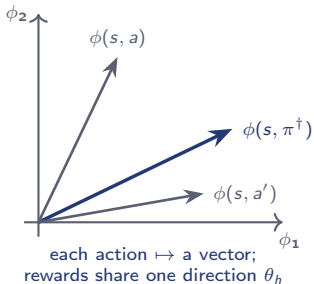
- ϕ = “what matters” about a state–action.
- θ_h = how features translate into reward.
 - **Reward poisoning:** $\theta_h \rightarrow \theta_h^\dagger$

Linear reward and dynamics:

$$\underbrace{r_h(s, a) = \phi(s, a)^\top \theta_h}_{\text{reward}}, \quad \underbrace{P_h(\cdot \mid s, a) = \sum_i \phi_i(s, a) P_h^{(i)}(\cdot)}_{\text{transitions}}.$$

Theoretical Setting: linear MDPs

Each “situation” (state and action) is mapped to a **feature vector** $\phi(s, a) \in \mathbb{R}^d$ ($\|\phi\|_2 \leq 1$).



Linear reward and dynamics:

$$\underbrace{r_h(s, a) = \phi(s, a)^\top \theta_h}_{\text{reward}}, \quad \underbrace{P_h(\cdot \mid s, a) = \sum_i \phi_i(s, a) P_h^{(i)}(\cdot)}_{\text{transitions}}.$$

Intuition:

- ϕ = “what matters” about a state–action.
- θ_h = how features translate into reward.
 - **Reward poisoning:** $\theta_h \rightarrow \theta_h^\dagger$
- Optimal value is linear too:
 $Q_h^*(s, a) = \phi(s, a)^\top w_h$.
- Tabular MDPs are the special case
 $\phi = 1_{(s, a)}$.
- **No-regret assumption:** learning algorithm’s regret is sublinear ($o(T)$).

Characterizing intrinsic robustness: CQP

We find (and prove sufficient and necessary conditions) that attackability reduces to a single **convex quadratic program** predicting whether an attacker can or cannot perturb rewards (new $\theta_h^\dagger$), making $\pi^\dagger$ look optimal.

CQP (Theorem 4.2)

$$\epsilon^* = \max_{\epsilon, \{\theta_h^\dagger\}} \epsilon$$

► how strongly can $\pi^\dagger$ win?

$\epsilon^* > 0 \Leftrightarrow$ **vulnerable**

$\epsilon^* \leq 0 \Leftrightarrow$ **intrinsically robust**

Characterizing intrinsic robustness: CQP

We find (and prove sufficient and necessary conditions) that attackability reduces to a single **convex quadratic program** predicting whether an attacker can or cannot perturb rewards (new $\theta_h^\dagger$), making $\pi^\dagger$ look optimal.

CQP (Theorem 4.2)

$$\epsilon^* = \max_{\epsilon, \{\theta_h^\dagger\}} \epsilon$$

► how strongly can $\pi^\dagger$ win?

$$\text{s.t. } Q_{\{\theta_h^\dagger\}}(s, \pi^\dagger(s)) \geq \epsilon + Q_{\{\theta_h^\dagger\}}(s, a)$$

► $\pi^\dagger$ beats every rival by a margin ϵ

$\epsilon^* > 0 \Leftrightarrow$ **vulnerable**

$\epsilon^* \leq 0 \Leftrightarrow$ **intrinsically robust**

Characterizing intrinsic robustness: CQP

We find (and prove sufficient and necessary conditions) that attackability reduces to a single **convex quadratic program** predicting whether an attacker can or cannot perturb rewards (new $\theta_h^\dagger$), making $\pi^\dagger$ look optimal.

CQP (Theorem 4.2)

$$\epsilon^* = \max_{\epsilon, \{\theta_h^\dagger\}} \epsilon$$

► how strongly can $\pi^\dagger$ win?

$$\text{s.t. } Q_{\{\theta_h^\dagger\}}(s, \pi^\dagger(s)) \geq \epsilon + Q_{\{\theta_h^\dagger\}}(s, a) \quad \text{► } \pi^\dagger \text{ beats every rival by a margin } \epsilon$$

$$\langle \phi(s, \pi^\dagger(s)), \theta_h^\dagger \rangle = \langle \phi(s, \pi^\dagger(s)), \theta_h \rangle \quad \text{► obeying } \pi^\dagger \text{ costs the attacker 0}$$

$\epsilon^* > 0 \Leftrightarrow$ **vulnerable**

$\epsilon^* \leq 0 \Leftrightarrow$ **intrinsically robust**

Characterizing intrinsic robustness: CQP

We find (and prove sufficient and necessary conditions) that attackability reduces to a single **convex quadratic program** predicting whether an attacker can or cannot perturb rewards (new $\theta_h^\dagger$), making $\pi^\dagger$ look optimal.

CQP (Theorem 4.2)

$$\epsilon^* = \max_{\epsilon, \{\theta_h^\dagger\}} \epsilon$$

$$\text{s.t. } Q_{\{\theta_h^\dagger\}}(s, \pi^\dagger(s)) \geq \epsilon + Q_{\{\theta_h^\dagger\}}(s, a)$$

$$\langle \phi(s, \pi^\dagger(s)), \theta_h^\dagger \rangle = \langle \phi(s, \pi^\dagger(s)), \theta_h \rangle$$

$$\|\theta_h^\dagger\|_2 \leq \sqrt{d}$$

► how strongly can $\pi^\dagger$ win?

► $\pi^\dagger$ beats every rival by a margin ϵ

► obeying $\pi^\dagger$ costs the attacker 0

► perturbation stays realistic

$\epsilon^* > 0 \Leftrightarrow$ **vulnerable**

$\epsilon^* \leq 0 \Leftrightarrow$ **intrinsically robust**

Developing optimal attacks

Role of No-Regret: The optimal attack strategy exploits the sublinear ($o(T)$) regret guarantee to convince the learner in at least $T - o(T)$ episodes of the optimality of $\pi^\dagger$ with cost $o(T)$.

Developing optimal attacks

Role of No-Regret: The optimal attack strategy exploits the sublinear ($o(T)$) regret guarantee to convince the learner in at least $T - o(T)$ episodes of the optimality of $\pi^\dagger$ with cost $o(T)$.

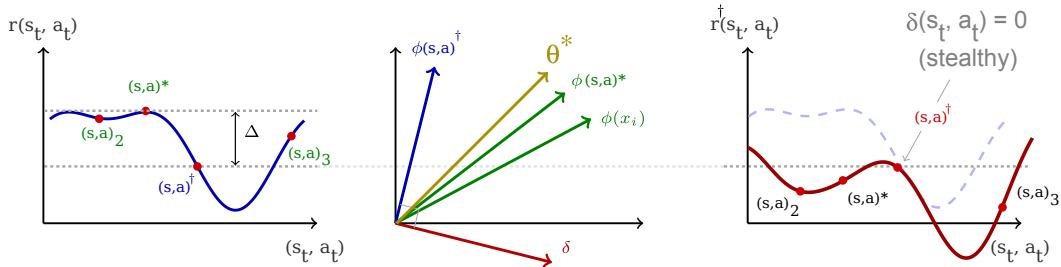


Figure: Left: Reward w.r.t. state-action. Center: Linear MDP mapping representation. Right: Reward after perturbation. Let $(s, a)^\dagger = (s, \pi^\dagger(s))$.

Developing optimal attacks

Inspired by our characterization, we develop two attacks using our CQP to construct adversarial poisoning under the following access levels:

Developing optimal attacks

Inspired by our characterization, we develop two attacks using our CQP to construct adversarial poisoning under the following access levels:

White-box (knows the model)

Only perturb when the learner *deviates* from $\pi^\dagger$; obeying is left untouched and free.

⇒ sublinear $o(T)$ total budget.

Black-box (must estimate it)

Stage 1: nudge the learner toward $\pi^\dagger$ while learning the model. **Stage 2:** apply a steady, targeted penalty to rivals.

⇒ still sublinear, $\tilde{O}(T^{3/4})$.

Same idea, different knowledge: pay only when the learner disagrees with the target, so the budget grows slower than the horizon.

Empirical demonstration of theoretical claims

We test our theoretical predictions on *classical Deep RL* problems from the MuJoCo/Gymnasium libraries. We implement our white-box, black-box, and baseline AT attacks against LSVI-UCB; we collect L2 policy difference and the adversarial budget used.

Empirical demonstration of theoretical claims

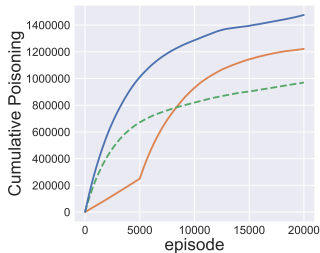
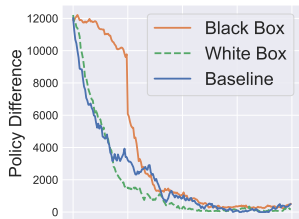
We test our theoretical predictions on *classical Deep RL* problems from the MuJoCo/Gymnasium libraries. We implement our white-box, black-box, and baseline AT attacks against LSVI-UCB; we collect L2 policy difference and the adversarial budget used.

1. Construct $\pi^\dagger$ (half-cheetah: run backwards, not forward).
2. Approximate linear MDP using Contrastive Representation Learning (CTRL+Soft AC).
3. Run attacks.

Empirical demonstration of theoretical claims

We test our theoretical predictions on *classical Deep RL* problems from the MuJoCo/Gymnasium libraries. We implement our white-box, black-box, and baseline AT attacks against LSVI-UCB; we collect L2 policy difference and the adversarial budget used.

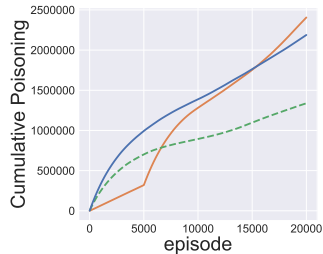
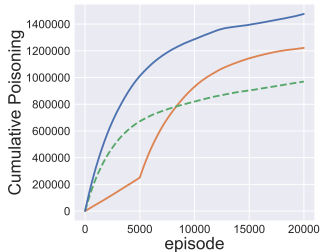
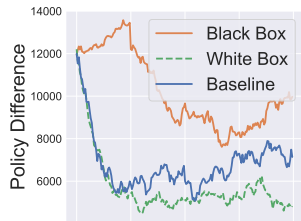
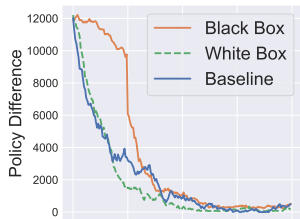
1. Construct $\pi^\dagger$ (half-cheetah: run backwards, not forward).
2. Approximate linear MDP using Contrastive Representation Learning (CTRL+Soft AC).
3. Run attacks.



Empirical demonstration of theoretical claims

We test our theoretical predictions on *classical Deep RL* problems from the MuJoCo/Gymnasium libraries. We implement our white-box, black-box, and baseline AT attacks against LSVI-UCB; we collect L2 policy difference and the adversarial budget used.

1. Construct $\pi^\dagger$ (half-cheetah: run backwards, not forward).
2. Approximate linear MDP using Contrastive Representation Learning (CTRL+Soft AC).
3. Run attacks.



Demonstration: Half-Cheetah

Left empty on purpose.

Takeaways

- **Intrinsic Robustness** is a **supplemental perspective** on reward poisoning, where the *environment setup* makes adversarial objectives impossible to attain under linear MDP assumptions.
- A **single convex test** ($\text{sign } \epsilon^*$) decides attackability in linear MDPs with *necessary and sufficient conditions*.
- Adversary fails under intrinsically robust environments **even against** non-robust algorithms.
- Results hold under learned linear representations of deep-RL benchmarks.

Code & full paper:
[github.com/aguilarjose11/
IntrinsicRobustness_ICML26](https://github.com/aguilarjose11/IntrinsicRobustness_ICML26)

Thank you!